Introducción

A partir de esta práctica hemos desarrollado y estudiado 2 modelos diferentes de colisión aplicados a sistemas de partículas. Dichos modelos se tratan del basado en velocidades y el basado en muelles. Además, se han incluido una serie de funciones bastante llamativas a ambas simulaciones de modelos que veremos conforme ahondemos en el desarrollo y, en el segundo caso, se ha aplicado 2 estructuras de datos diferentes para tratar de hacer el sistema de colisión mucho más óptimo. Este documento recoge tanto una pequeña documentación y descripción de las simulaciones realizadas, como algunos estudios y conclusiones obtenidas en base a los propósitos de laboratorio sobre el impacto computacional y/o estabilidad de los sistemas.

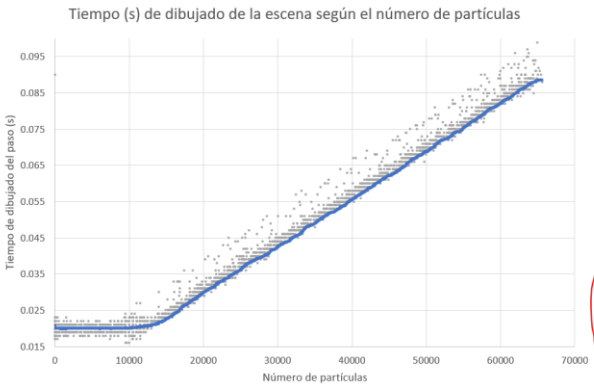
La primera simulación, con el modelo de velocidades, es la de un billar francés, aunque las texturas utilizadas son de un billar común y, de hecho, se ha tratado de implementar parcialmente la posibilidad de jugar a este último juego, y la segunda, una cubeta alrededor y dentro de la que se generan partículas que tratarán de imitar diversos fluidos utilizando el modelo y las ventajas señaladas.

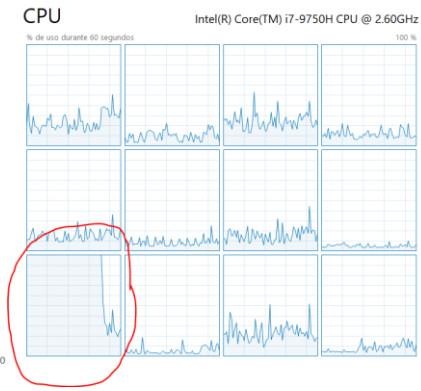
Detalles del computador de pruebas

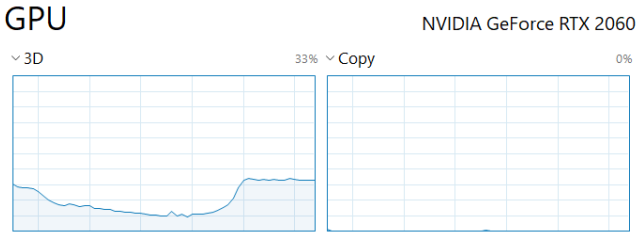
Para las pruebas se ha utilizado un computador MSI GL65 95EK con los siguientes detalles:

- Configuración mediante el driver de NVIDIA para el uso específico de la tarjeta gráfica dedicada: NVIDIA RTX 2060. Se ha comprobado a través del administrador de tareas que la carga de dibujado que supone la simulación aparece reflejada únicamente en dicho procesador.
- Uso de Windows 10, lo que supone la máxima compatibilidad con el SW de NVIDIA.
- Simulación realizada en el monitor secundario conectado por HDMI (por pruebas realizadas anteriormente, a veces implica una reducción del rendimiento por el acceso a la zona de memoria del mismo).

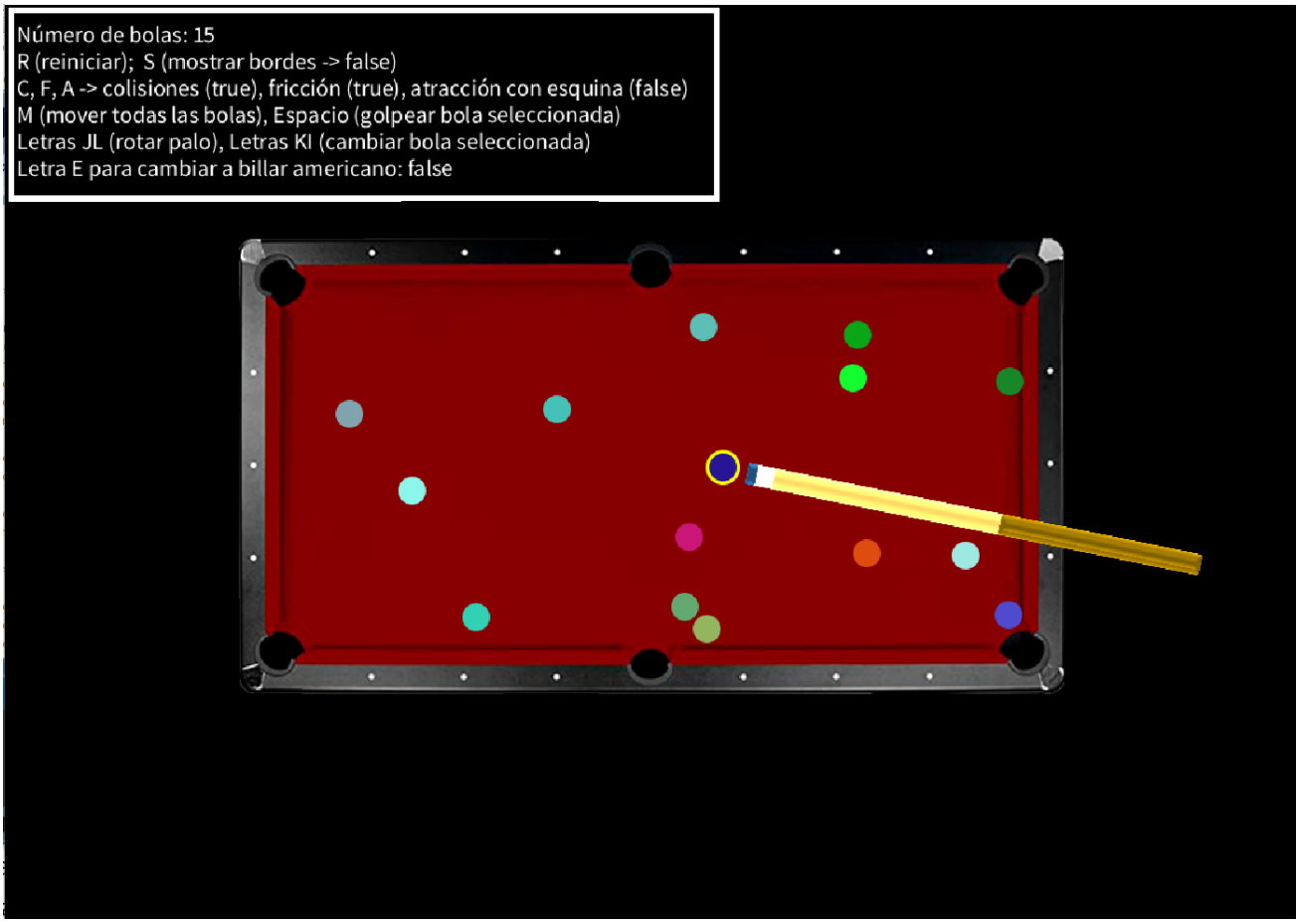
Es de destacar lo comentado en la práctica anterior: es posible que, debido a que Processing para utilizar solo un núcleo de la CPU para el proceso de la simulación en mi equipo, se produzca un cuello de botella que implique tiempos y una curva poco concluyente a la hora de realizar las gráficas. Los resultados fueron estos la última vez para el sistema de partículas de fuegos artificiales (en la práctica anterior se puede visualizar una versión más grande de estas gráficas).







Modelo de colisión por velocidades



Como se puede apreciar en la captura, en esta simulación contamos con una amplia gama de opciones interactivas. La simulación comienza con un cierto movimiento de las bolas, cuyo número es seleccionable desde código en las primeras líneas del main y su color es aleatorio. El usuario puede decidir mover todas con M, generar una atracción hacia la esquina inferior derecha, o seleccionar una y golpearla con fuerza aleatoria rotando el palo en la dirección que decida con el resto de controles descritos.

Incluso, puede tratar de jugar al billar americano activando dicha opción, aunque presenta algunos bugs con el recuento y los cestos intermedios a los planos frontera no están disponibles; sin embargo, se ha preferido mantener esta opción para su evaluación a fin de mantener una perspectiva del trabajo realizado.

```
void planeCollision(ArrayList<PlaneSection> planes)
{
  for(PlaneSection p: planes)
  {
    PVector n = p.getNormal(), point = p.getPoint1(),
      x = PVector.add(_s,n.copy().rotate(HALF_PI).mult(_radius));
    float dcol = PVector.dot(PVector.sub(x,point).normalize(),n);

    PVector nearest;
    //dcol:
    //0 significa que, _s forma parte del plano, 1 significa que _s esta en el plano

    if(selected_americano)
    {
      PVector point2 = p.getPoint2();
      float min_dist = _radius;
      if(PVector.dist(_s, point) < min_dist || (PVector.dist(_s, point2) < min_dist))
      {
        _destroy = true;
        _ps.reduceParticles();
        return;
      }
    }
```

El modelo desactiva una booleana de las bolas para no dibujarlas y que no colisionen si tocan las esquinas.

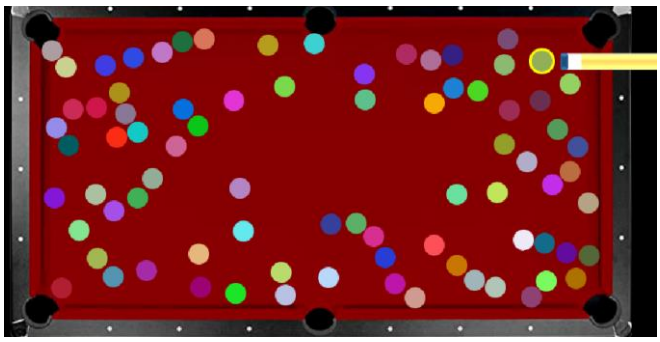
En cuanto a las colisiones, se implementan por separado dos formas del modelo de velocidades cuyas colisiones, en ambos casos y mediante diferentes constantes, cuentan con pérdida de energía por colisión:

- Un modelo partícula-plano para simular la que existe entre las bolas y las fronteras del billar.
- El mismo aplicado entre partículas.

Si bien podríamos ahondar bastante en los detalles de dicho modelo, es conveniente centrarse en los resultados de la implementación, cuestiones planteadas y consecuentes mejoras en la implementación.

Se ha realizado diferentes pruebas con esta simulación tratando de cambiar el número de bolas simultáneas, cuántas en movimiento, la existencia de fricción o la inclusión de la fuerza constante hacia la esquina, y en todos los casos las conclusiones en cuanto a la estabilidad del sistema son las mismas:

1. A mayor cantidad de movimiento en la escena y menos espacio para sus "partículas" más colisiones encontramos y con más transmisión de fuerza. Si bien el sistema partícula-partícula soporta adecuadamente colisiones de 2 y hasta 3 bolas seguidas, debemos de recordar que cada colisión implica un reposicionamiento de la geometría y un recálculo de la velocidad en sentido contrario. Si dicha partícula se encuentra rodeada por otras que en ausencia de esta en la posición que debería de estar, porque se encuentra solapada por un instante con la que está haciendo colisión, han ocupado su posición o cercanías, el reposicionamiento se puede traducir fácilmente en nuevos solapamientos que comiencen a desplazar todo el entorno de partículas inesperada y rápidamente, generando artefactos como parpadeos y cambios de ubicación imposibles.



2. El sistema partícula-plano ya de por sí contaba con un problema: el tunneling. La implementación original se basa en examinar la inclusión del radio de las esferas en el segmento del plano y desplazarlo hacia afuera en la dirección de la normal utilizando esta distancia. Esto no tiene en cuenta grandes velocidades que puedan provocar atravesar completamente el área del plano, velocidades como las que se pueden generar en un sistema agarrotado de partículas, con muchas colisiones o sin fricción que las disipe poco a poco, o con el problema recién mencionado.

Es por eso que cualquier acción entre las siguientes: añadir más esferas, eliminar la fricción, añadir más velocidades, aumentar el número de colisiones… No van a hacer más que forzar e inestabilizar el sistema.

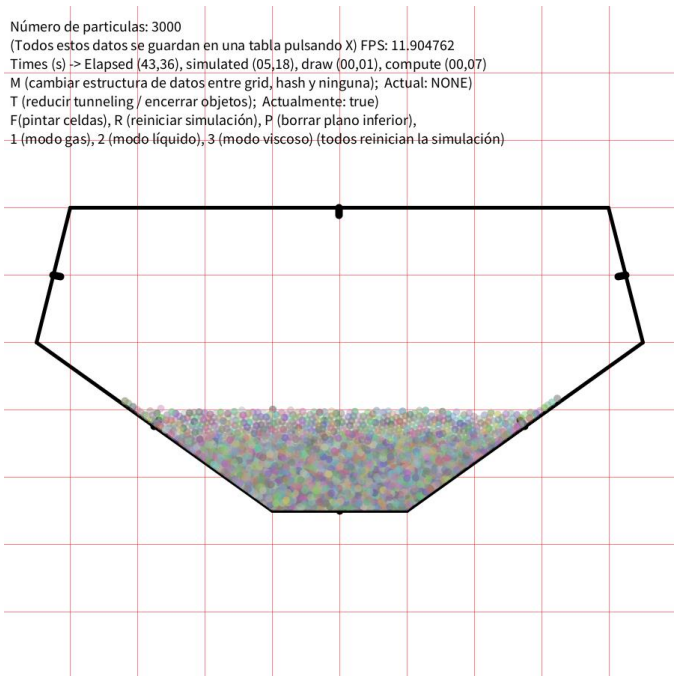
```
if (dcol < 0)
{
  nearest = getPlaneNearest(p);
  correctDirPos(nearest,n);
}

if(abs(dcol) < 0.25)
{
  nearest = getPlaneNearest(p);
  if(PVector.dist(nearest, _s) < _radius) correctDirPos(nearest,n);
}
```

Como medida preventiva al tunneling, aunque no evita ciertos parpadeos, se han planteado dos estrategias en cuanto a la colisión partícula-plano.

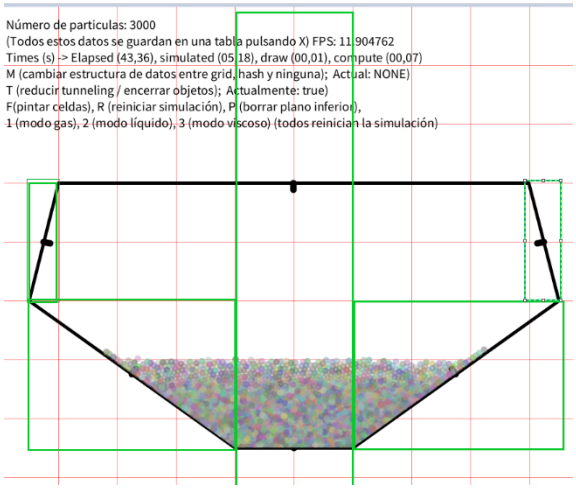
Siendo el valor "dcol" (en este caso) la proyección de un vector cualquiera desde la posición más cercana de la bola al plano (se utiliza la dirección de la normal y el radio para obtener un punto x desplazado del centro de la esfera) hacia un punto 'point' del plano sobre la normal hacia el interior del billar, este se puede traducir como la distancia perpendicular del centro de la esfera al plano. Es en esto en lo que ya se basa este modelo, comparando la distancia con el radio de la esfera. Pero existe otro aspecto relevante: el signo. Se puede utilizar el signo para adivinar si la esfera esta saliendo o esta fuera del billar, y eso es lo que se ha utilizado para volver a introducir casi cualquier esfera que intente salir.

Por otro lado. El reposicionamiento es parcialmente diferente al utilizado en ejercicios de clase. En este caso, se calcula el punto proyección del centro de la esfera sobre el plano, y se parte de ese punto para actualizar la posición. En varios intentos ha demostrado ser un pelín más efectiva que la implementación basada enteramente en la distancia "dcol" desde el centro, aunque las cosas cambian en la segunda simulación.

Modelo de colisión por muelles



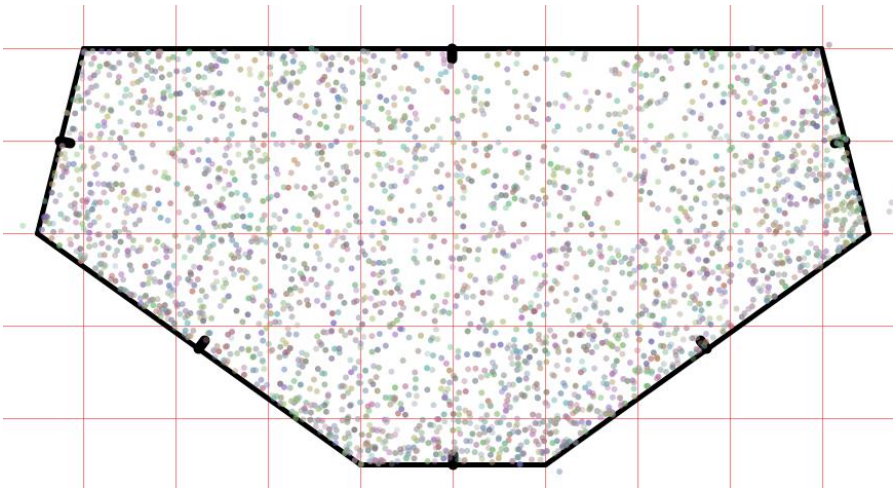
Para el caso lo primero que me gustaría destacar es la dificultad de implementar el sistema de colisión partícula plano en un escenario como este. Si bien se han introducido diversos elementos para evitar errores de tunneling y errores en la dirección de las normales, al estar manejando tal cantidad de partículas de tan pequeño tamaño es muy difícil evitar el escape de varias de ellas en el ejemplo que vamos a ver, de forma que en este caso he recurrido al modelo de reposicionamiento y ldir (desde el centro de la partícula y no desde el extremo más cercano) con el único fin de simplificar el código y pulir errores. Gracias a ello me pude dar cuenta de que el principal problema se encontraba en que, en este caso, por los ángulos agudos que forman entre sí los planos, es muy necesario delimitar la zona de actuación de cada plano a la hora de calcular las colisiones, y por ello he incluido esta función que evalúa si la partícula se encuentra dentro del cubo de los planos inclinados, o en la zona delimitada por los planos rectos.



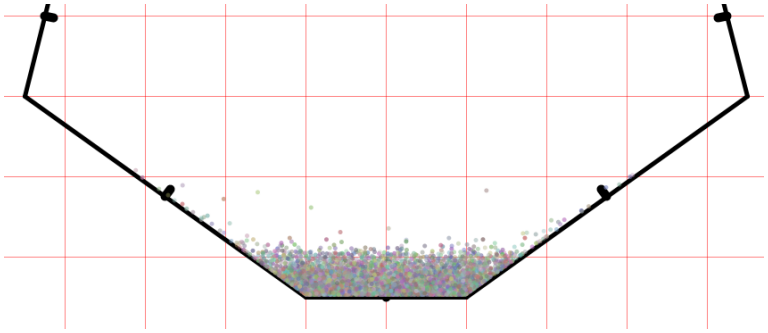
En cuanto al tunneling, recurrimos al mismo método desarrollado anteriormente, pero con la posibilidad de activarlo y desactivarlo. Al estar activado, la mayor parte de partículas se mantienen en el interior de la cubeta, pero además, las partículas que escapen, incluidas las que salgan por el plano inferior abierto, volverán a entrar de nuevo, por lo que si se pretende probar el efecto embudo o realizar pruebas de computo, puede ser hasta conveniente para mantener la mayor parte de partículas siendo dibujadas en el área de la pantalla, aunque la implementación no sea realista para otras utilidades.

En esta simulación podemos elegir entre 4 modelos diferentes de fluido: el que se encuentra seleccionado por defecto, y otros 3 que asemejan:

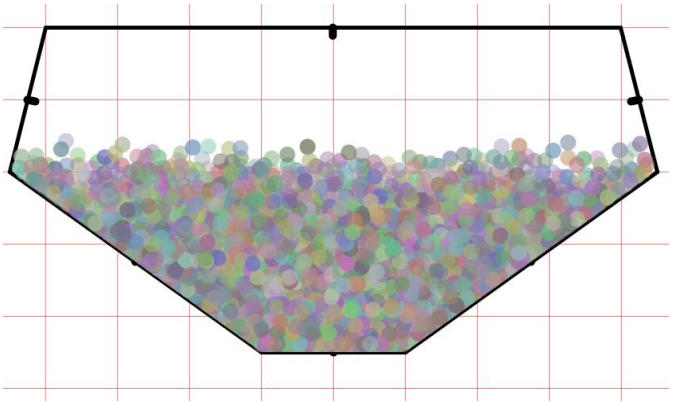
- Un gas. Constando de una gran propiedad elástica, poca fricción, pequeño tamaño y una distancia inicial entre partículas ínfima para asemejar nada más invocar el sistema la expansión del gas por toda la cámara.



- Un líquido. También de partículas pequeñas, pero de una propiedad elástica mucho menor, más separación, más peso y más fricción que mantienen las partículas menos movidas y más unidas y pesadas pero generando olas y diversos efectos en la superficie cuando se amontonan, hasta llegar a la calma.



- Un elemento viscoso. Con partículas mucho más grandes, la mayor constante de rozamiento, una separación entre partículas ínfima para superponerlas y generar un fluido lento y grumoso.

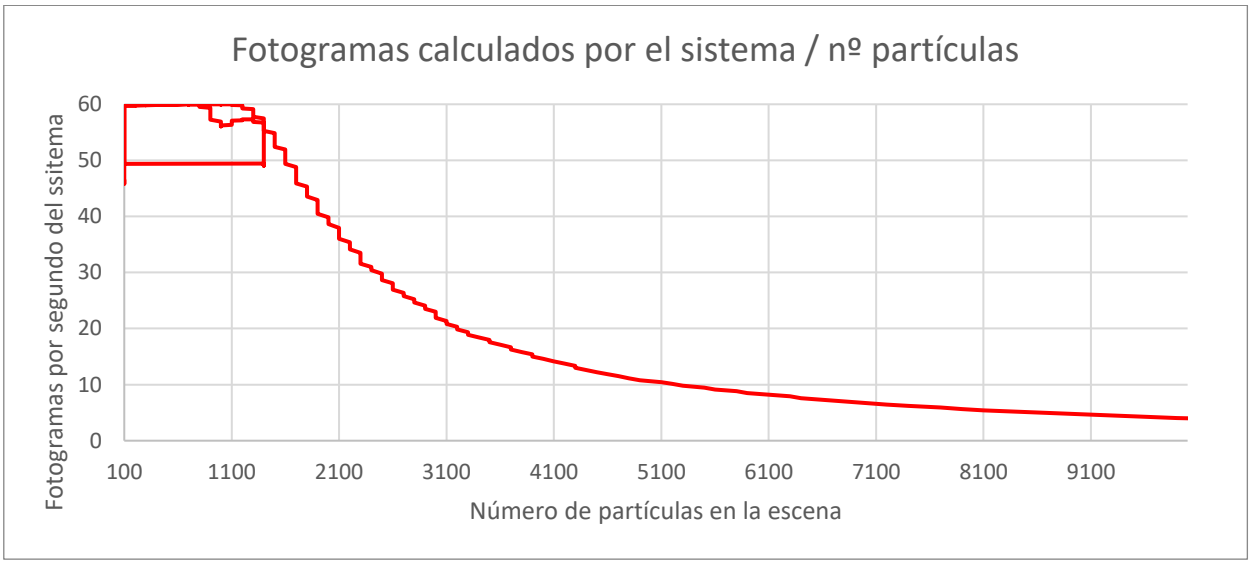


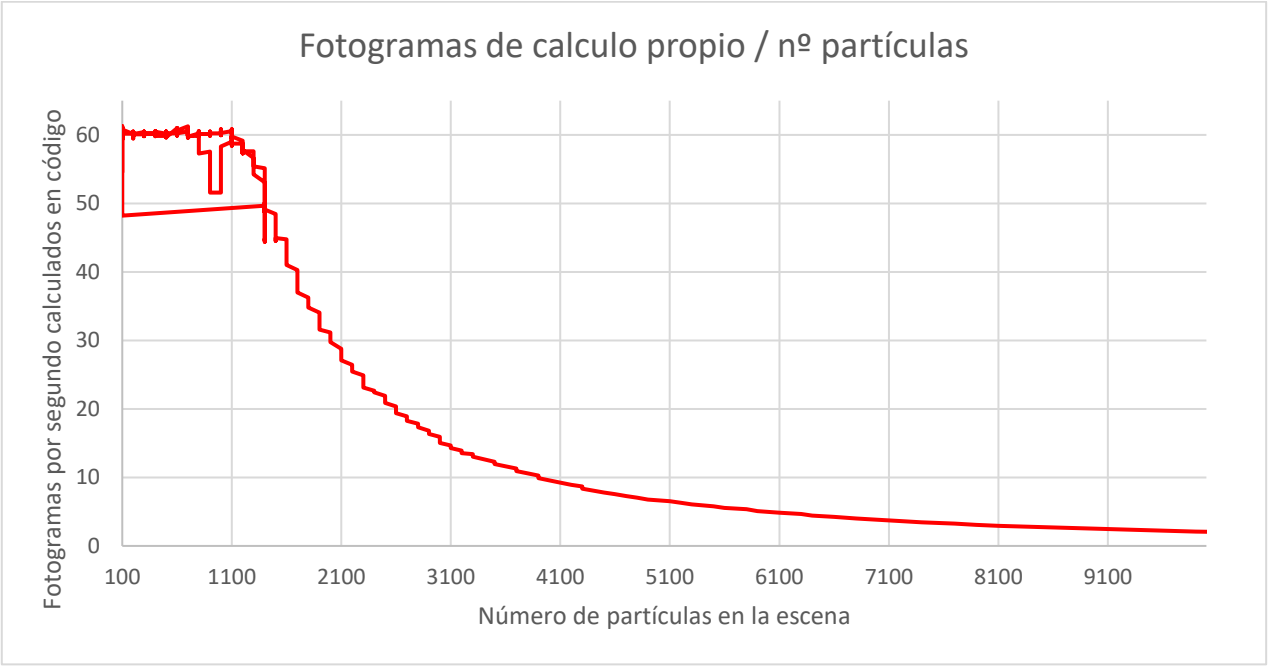
Para las pruebas de computo que vamos a realizar puede ser muy buena opción el gas. Y para ellas lo que vamos hacer es comparar la velocidad en fotogramas, el tiempo de computo de las colisiones y de la integración y el tiempo total incluyendo el dibujado desde 100 a 10000 esferas añadiendo de 100 en 100 para cada sistema de los utilizados: colisiones a fuerza bruta, grid y tabla hash, utilizando para estas dos últimas 3 tamaños de celda distintos: 10, 20 y 60.

Comparativa de rendimiento 1: A lo bruto vs. Estructuras de datos

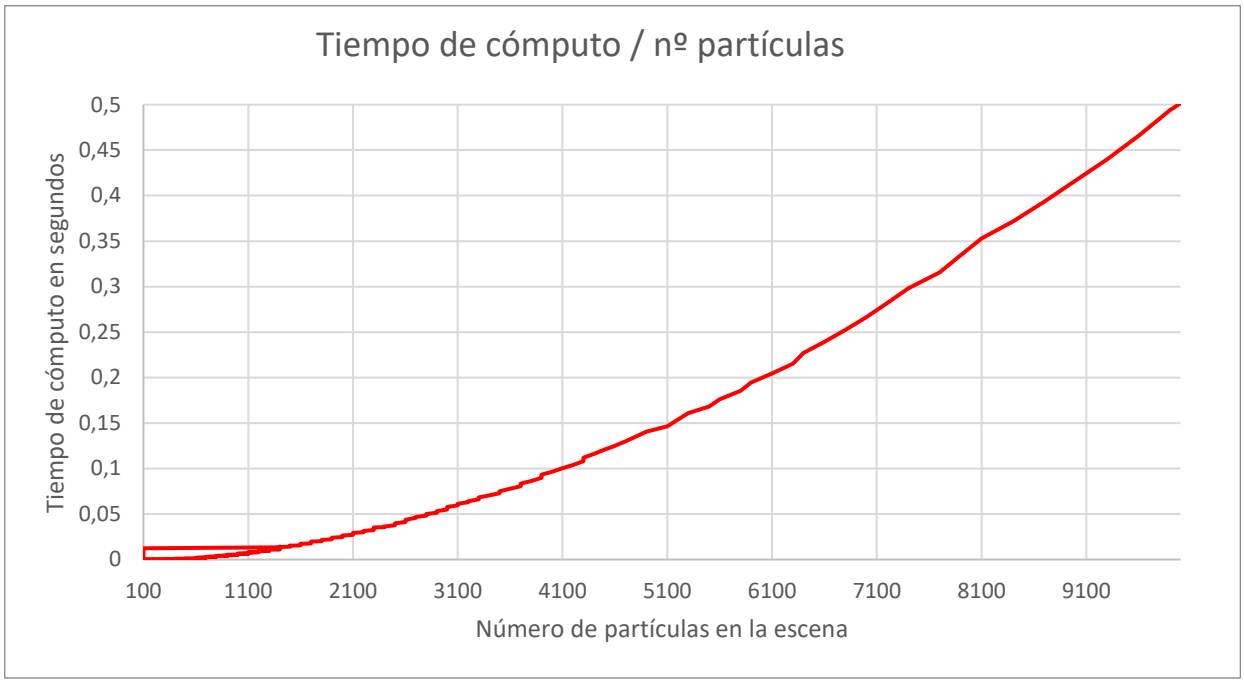
Las primeras gráficas que vamos a visualizar son bastante esclarecedoras en cuando a la diferencia de rendimiento que supone el uso de estructuras de datos que subdividan el espacio para reducir el número de hipotéticas interacciones, y el no hacerlo. Hacemos uso de 4 medidas: un cálculo de fotogramas propio de Processing, uno realizado en el código de la aplicación, el tiempo de computo sin considerar dibujado y el tiempo total que sí lo considera.

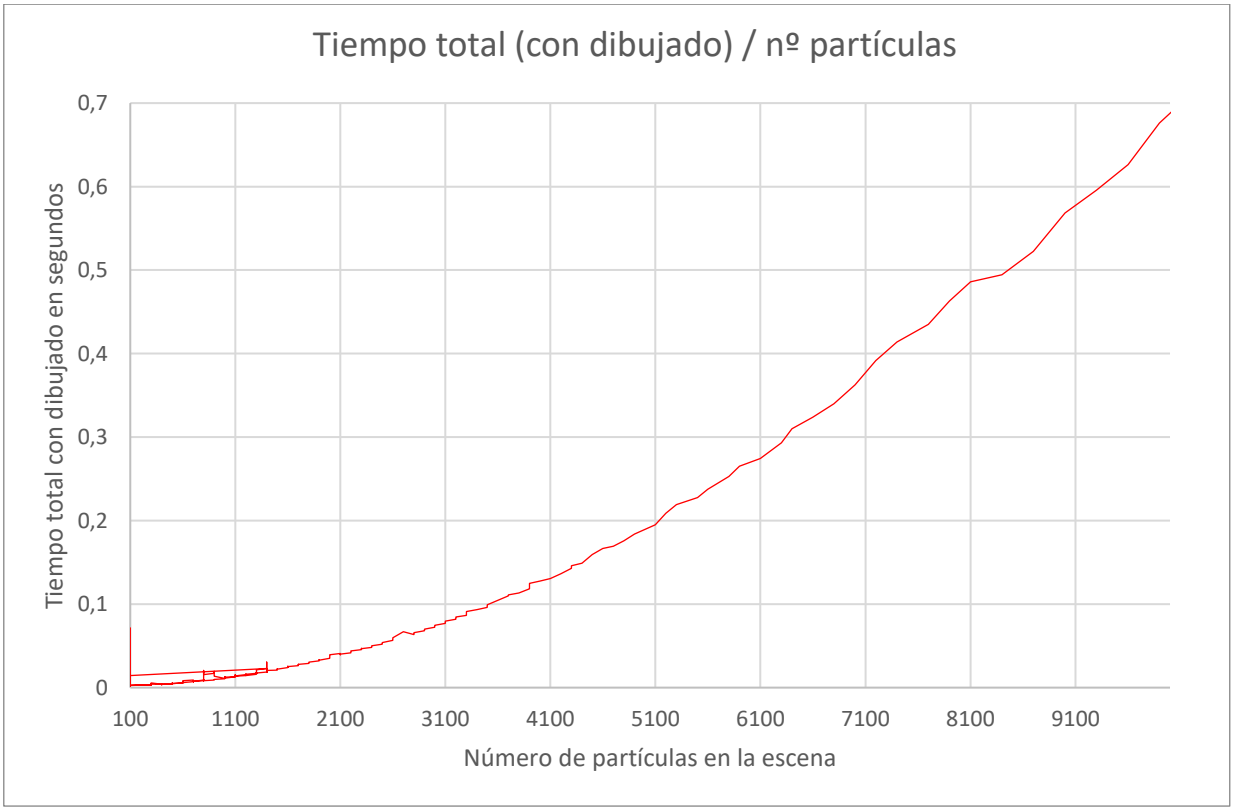
Sistema de partículas sin Hash ni Grid





Efectivamente, el rendimiento en fotogramas por segundo sin estructuras de datos disminuye exponencialmente, y de la misma forma aumentan tanto el tiempo de cómputo como el total de dibujado.



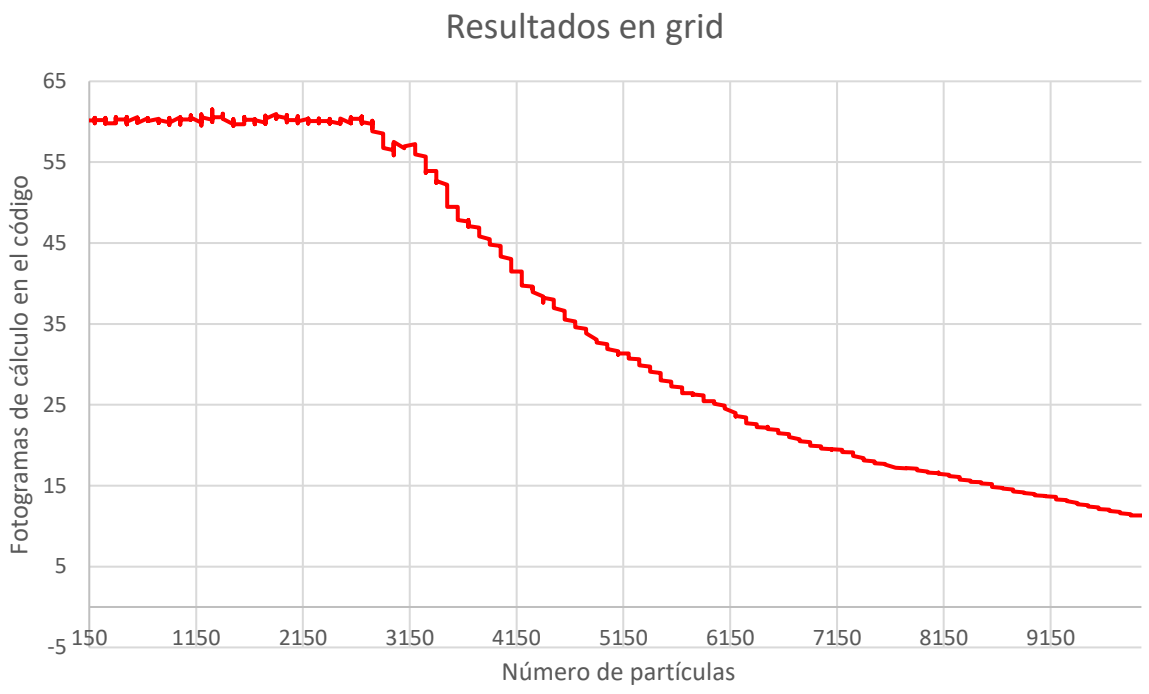


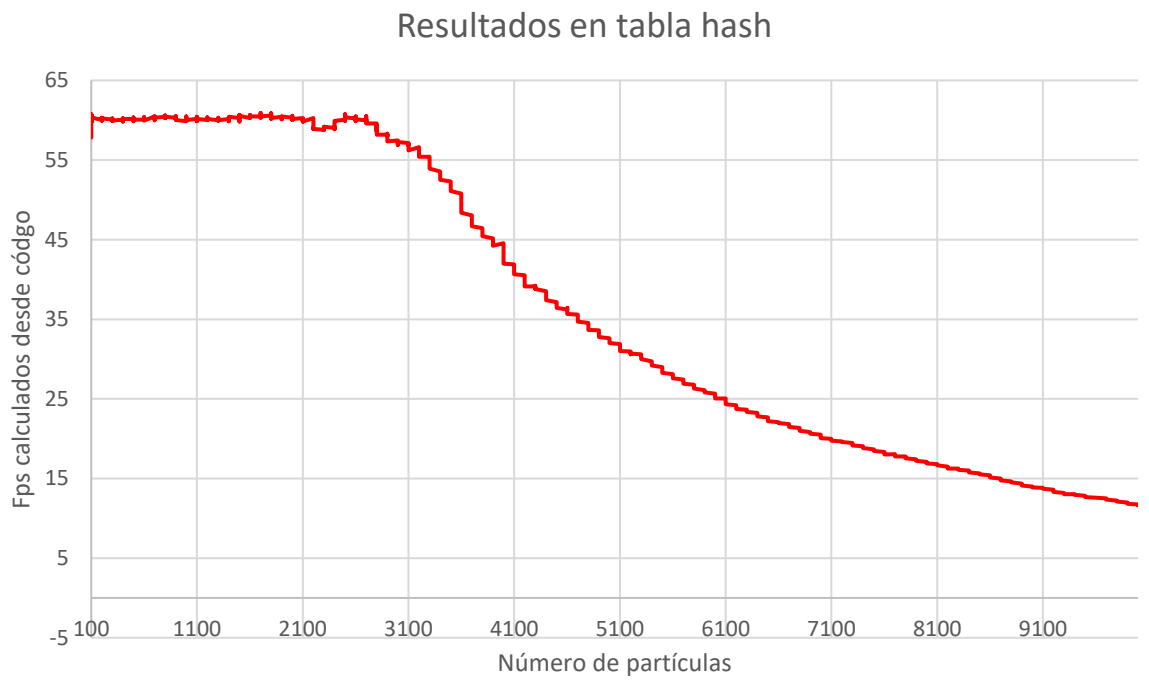
Las gráficas utilizadas toman numerosas muestras de un mismo número de partículas, al mismo tiempo que el trazado proviene de una media móvil de 24 muestras del mismo número o similar. Es un resultado bastante cercado.

A continuación se muestran juntas las gráficas de cada una de las estructuras de datos, Grid y Hash; sin embargo, se descartarán las gráficas de cómputo de fps por parte del Processing, pues muestran información demasiado similar, e incluso menos precisa para generación de gráficas, que nuestro cálculo dentro del código.

El primer escenario es el de un grid y tabla hash de 10x10

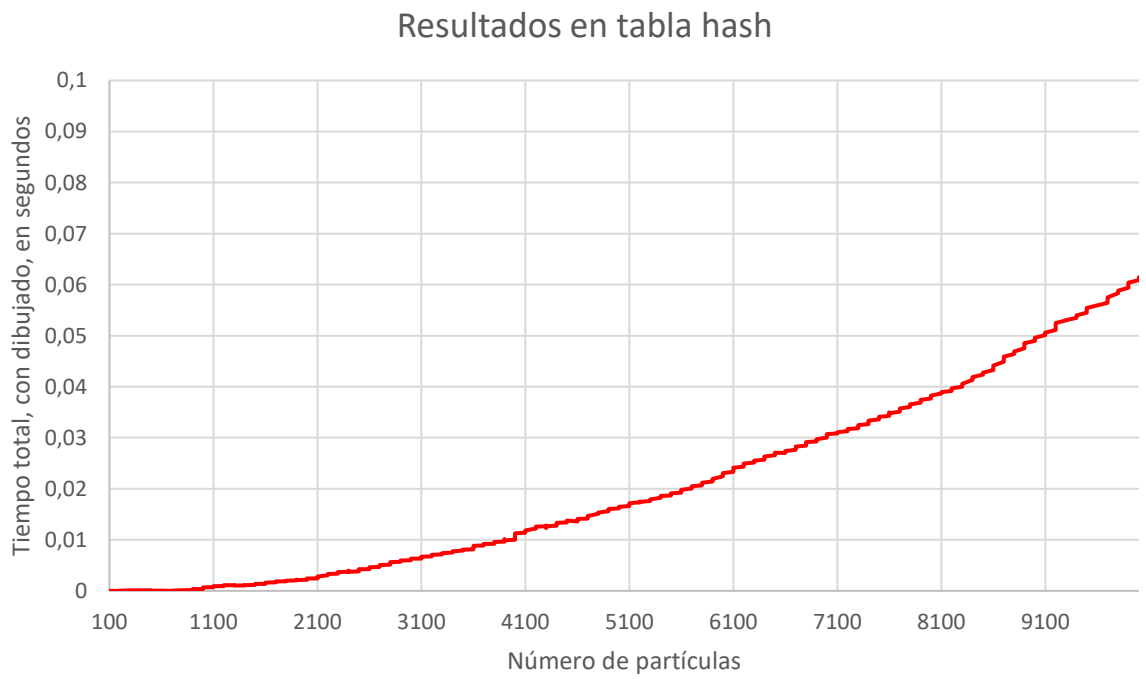
Fotogramas / nº partículas en





Computo sin dibujado

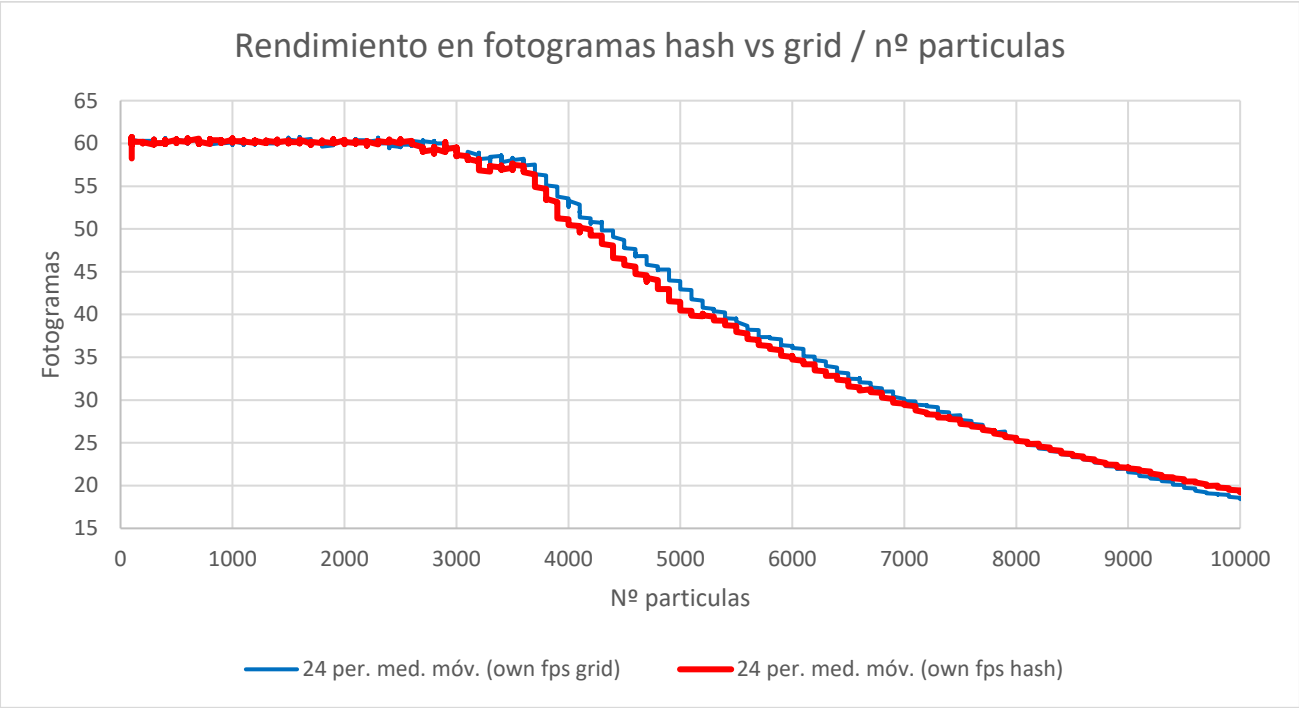
Cómputo total

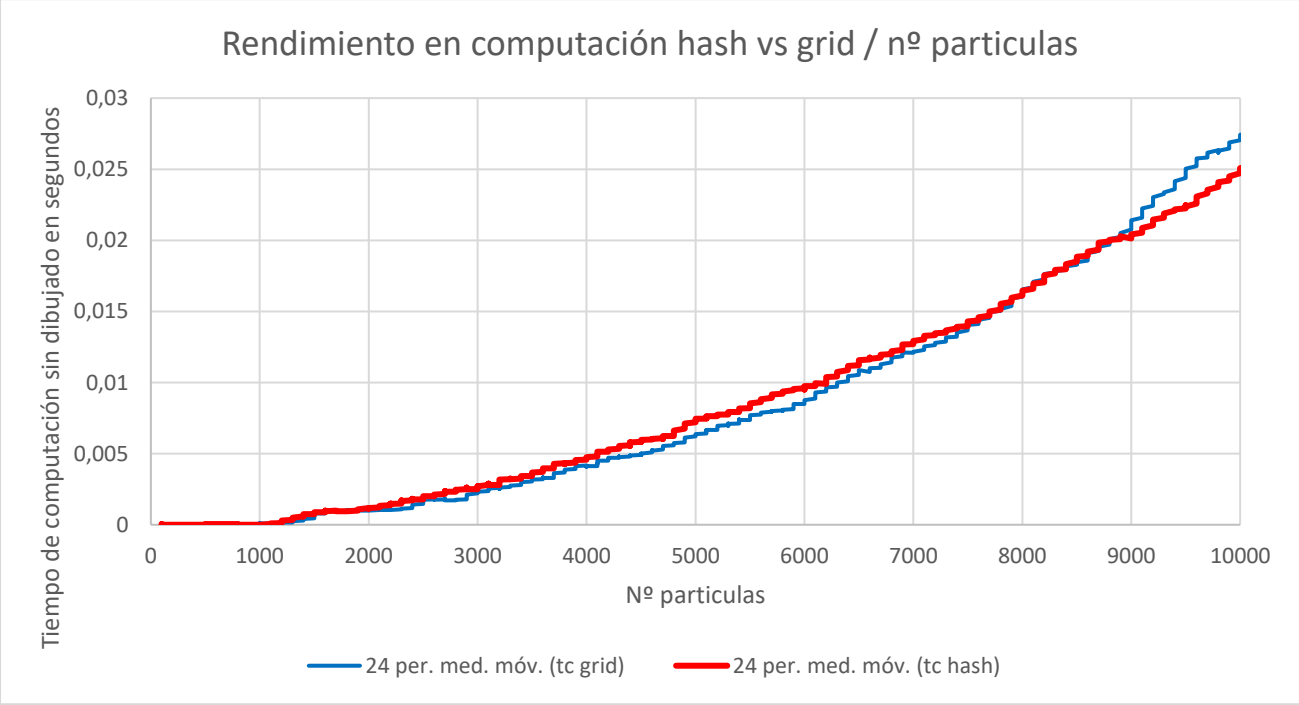


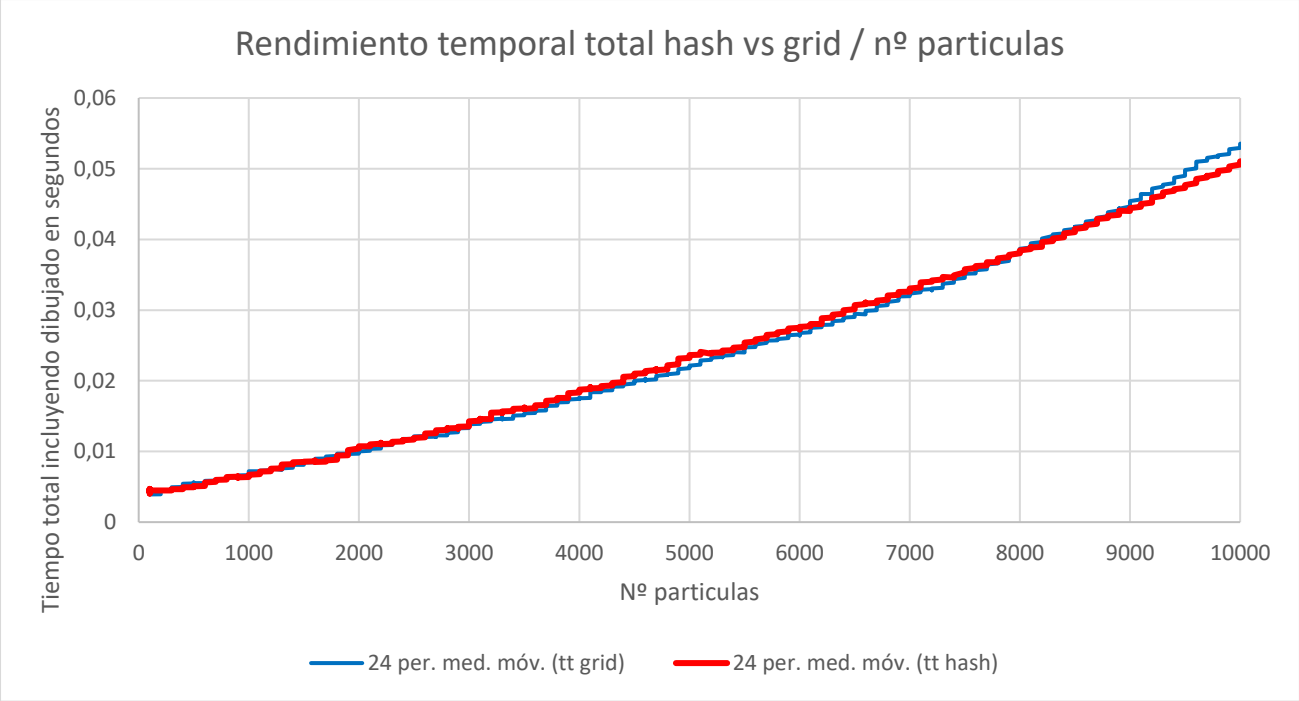
Si revisamos bien los tiempos, nos fijamos en que el rendimiento en este caso es ligeramente superior cuando se utiliza la tabla hash. Esta es una herramienta que tiene como principal objetivo ahorrar en memoria: un grid es una estructura de datos algo más sencilla que permite acceder de forma sencilla a elementos adyacentes, pero implica registrar todas las localizaciones de la escena incluidas las vacías, desaprovechando memoria. La

escena en la que estamos trabajando incluye toda su geometría activa de forma muy localizada en el centro, estando los alrededores vacíos, por lo que una tabla hash con no excesiva cantidad de espacio puede ser una gran opción para subdividir los eventos de nuestra escena como las colisiones; sin embargo, ahora vamos a ver cómo esta ventaja se desdibuja conforme añadimos espacios. Vamos a comprar, ahora más específicamente, grid y hash para los casos de las subdivisiones de 20x20 y 60x60 para ambas estructuras:

Grid y tabla hash de 20x20

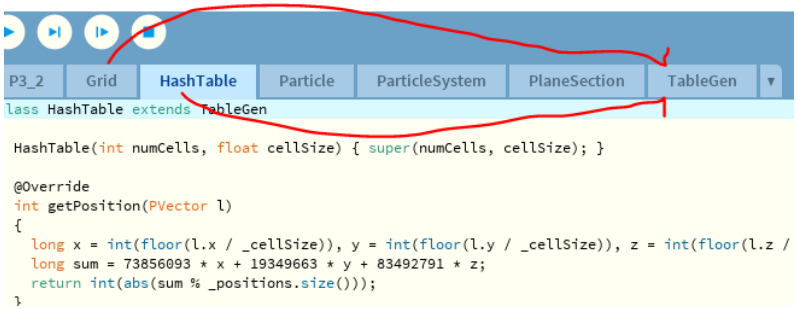






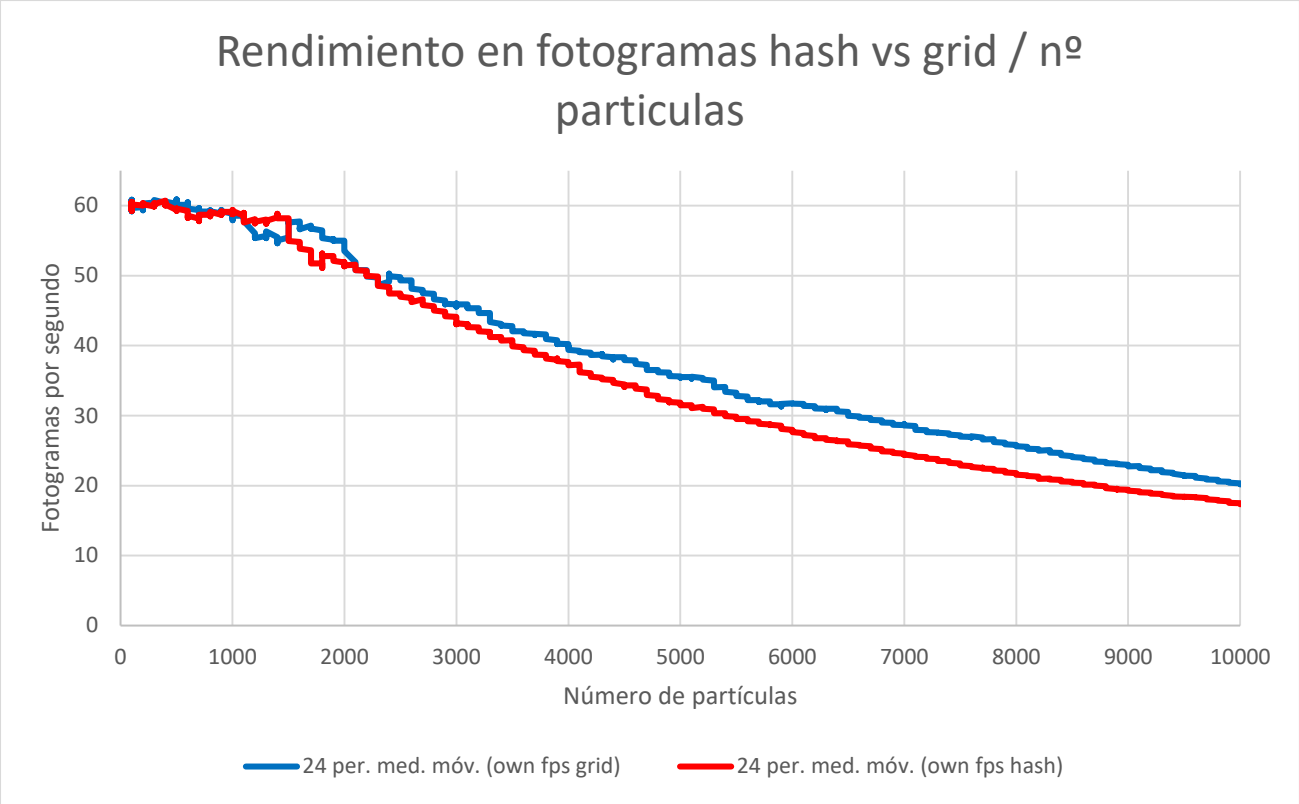
La ventaja de la tabla hash, efectivamente, se desdibuja, aunque sigue siendo una buena estructura que utilizar. Es posible que los resultados vistos puedan deberse a ciertos errores o excesos en la implementación de las estructuras; sin embargo, es destacable que no hay, en realidad, mucha diferencia entre ellas, pues en su formulación se ha decidido hacer uso de las capacidades de herencia y polimorfismo de java:

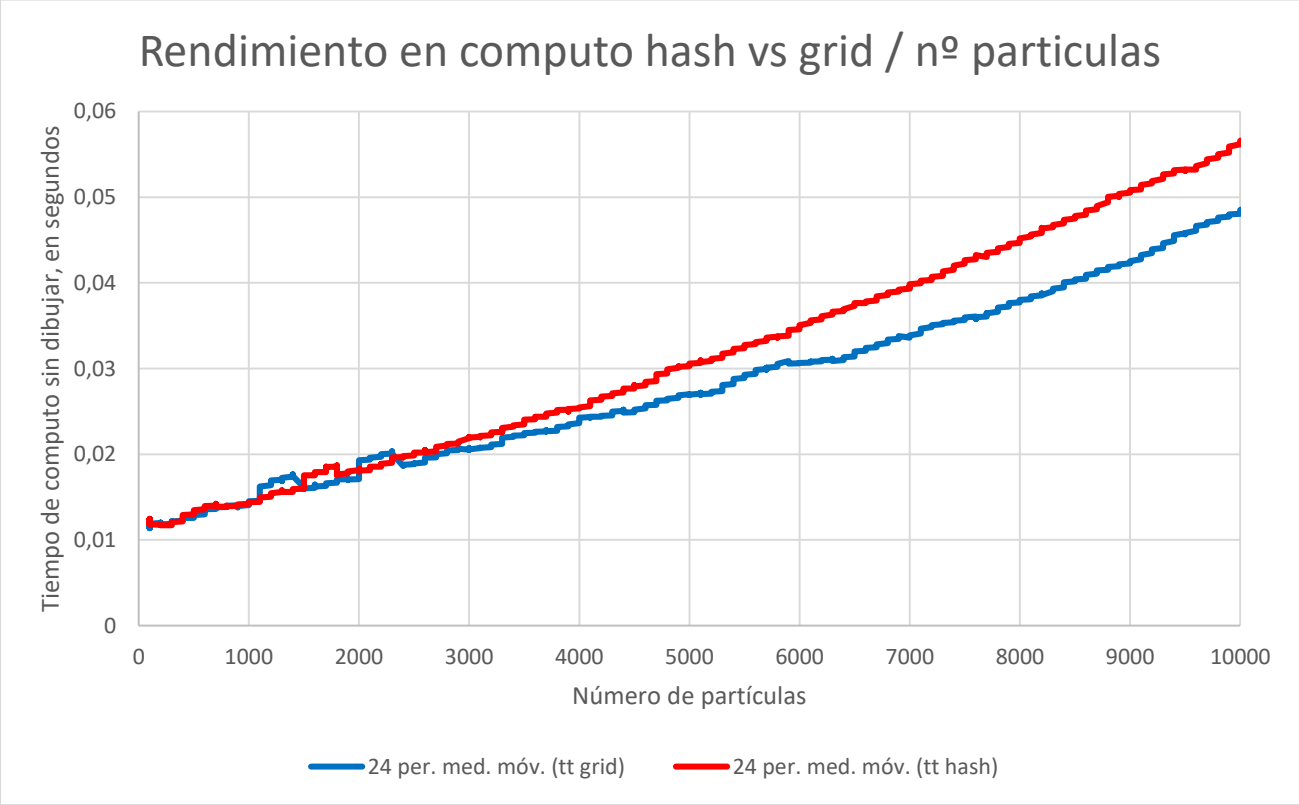
De esta forma, una de las mayores diferencias entre ambas estructuras es el método getPosition(), que en un caso es un acceso a determinada posición de una lista asemejando una matriz pero por cálculo, y en el otro caso se utiliza **la dispersión**: una serie de cálculos no excesivamente complejos, pero que ya suponen una diferencia frente al mero acceso. Cuantos más espacios añadimos a una estructura como la tabla hash, más compleja se vuelve, volviéndose más tedioso el uso, que es intensivo, de este tipo de métodos.

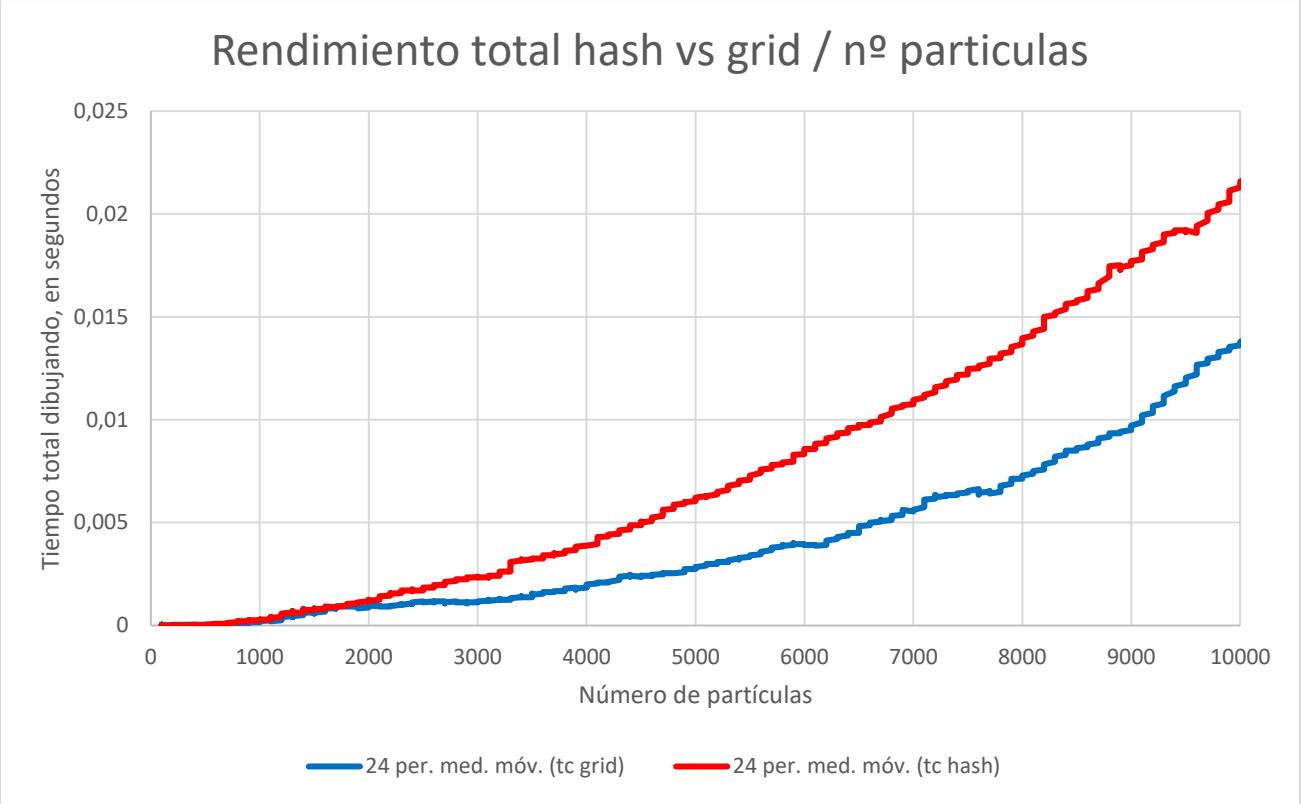


Es así que, cuando decidimos realizar una subdivisión del entorno mucho más amplia, como 60x60, los resultados son más claros:

Grid y tabla hash de 60x60







La tabla hash comienza a ser algo más ineficiente: la mayor parte de la geometría se encuentra muy localizada en la escena y no son necesarias tantas subdivisiones considerando la constante interacción de todas las partículas y el coste de uso, generación de vecinos… Cuando se trata de acceder a los elementos en Hash.